

# Helix OTA

## Helix OTA — Deployment & Container Orchestration Guide

**Document ID:** HELOTA-DEPLOY-001 **Version:** 1.0.0  
**Status:** Active **Last Updated:** 2026-03-05  
**Constitution Reference:** HelixConstitution v1 §1-§4  
**Target Platform:** Android 15 on Orange Pi 5 Max (RK3588)

---

### Table of Contents

1. [Infrastructure Overview](#)
  2. [Container Architecture](#)
  3. [Docker Compose Configuration](#)
  4. [Kubernetes Deployment](#)
  5. [Environment Configuration](#)
  6. [Networking](#)
  7. [Storage](#)
  8. [Scaling](#)
  9. [Monitoring](#)
  10. [Disaster Recovery](#)
  11. [vasic-digital/containers Integration](#)
-

# 1. Infrastructure Overview

## 1.1 Service Inventory

The Helix OTA platform consists of seven containerized services that together form a complete, enterprise-grade Over-The-Air update delivery system. Each service is independently deployable, horizontally scalable, and observably instrumented through the vasic-digital/observability submodule.

| # | Service                    | Image                     | Purpose                                                                                                       | CPU              | Memory              | Storage                   |
|---|----------------------------|---------------------------|---------------------------------------------------------------------------------------------------------------|------------------|---------------------|---------------------------|
| 1 | <b>helix-ota-server</b>    | helix-ota/server:1.0.0    | Go REST API server: artifact management, device registry, rollout engine, telemetry ingestion, authentication | 1 core (burst 2) | 512 Mi (burst 1 Gi) | 100 Mi (config + certs)   |
| 2 | <b>helix-ota-dashboard</b> | helix-ota/dashboard:1.0.0 | React 18 SPA served by nginx: admin UI for artifact upload, rollout management, fleet monitoring              | 0.25 core        | 128 Mi              | 50 Mi (static assets)     |
| 3 | <b>postgresql</b>          | postgres:16-alpine        | Primary relational database: device registry, artifacts, rollouts, telemetry events, audit log                | 2 cores          | 2 Gi                | 50 Gi (persistent volume) |
| 4 | <b>redis</b>               | redis:7-alpine            | L1/L2 cache: device check-in deduplication, rollout percentage cache, session                                 | 0.5 core         | 512 Mi              | 1 Gi (AO persistent)      |

| # | Service        | Image                    | Purpose                                                                                                                      | CPU         | Memory | Storage                        |
|---|----------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------|--------|--------------------------------|
| 5 | <b>minio</b>   | minio/<br>minio:latest   | store, rate<br>limit counters<br>S3-compatible<br>artifact<br>storage: OTA<br>zip binaries,<br>signatures,<br>delta payloads | 1<br>core   | 1 Gi   | 500 Gi<br>(artifact<br>volume) |
| 6 | <b>vault</b>   | hashicorp/<br>vault:1.15 | Secret<br>management:<br>artifact<br>signing key<br>(Transit<br>engine), TOTP<br>encryption<br>key, database<br>credentials  | 0.5<br>core | 256 Mi | 100 Mi<br>(Raft<br>storage)    |
| 7 | <b>traefik</b> | traefik:v3.0             | Reverse<br>proxy/ingress:<br>TLS<br>termination,<br>mTLS<br>passthrough,<br>route-based<br>routing,<br>metrics<br>collection | 0.5<br>core | 256 Mi | 50 Mi<br>(config +<br>certs)   |

### 1.2 Minimum Production Requirements

| Resource          | Minimum | Recommended | Notes                                                            |
|-------------------|---------|-------------|------------------------------------------------------------------|
| <b>CPU Cores</b>  | 4       | 8           | Server + PostgreSQL are CPU-intensive during artifact validation |
| <b>RAM</b>        | 8 Gi    | 16 Gi       | PostgreSQL buffer pool, Redis cache, server goroutine pools      |
| <b>Disk (SSD)</b> | 100 Gi  | 500 Gi      | Artifact storage                                                 |

| Resource       | Minimum | Recommended | Notes                                                                                      |
|----------------|---------|-------------|--------------------------------------------------------------------------------------------|
|                |         |             | dominates;<br>plan for<br>OTA zips at<br>1-2 Gi each                                       |
| <b>Network</b> | 1 Gbps  | 10 Gbps     | Fleet<br>downloads<br>can<br>saturate<br>links;<br>consider<br>CDN for<br>large fleets     |
| <b>Nodes</b>   | 1       | 3           | Minimum 3<br>for<br>Kubernetes<br>HA; single-<br>node<br>acceptable<br>for dev/<br>staging |

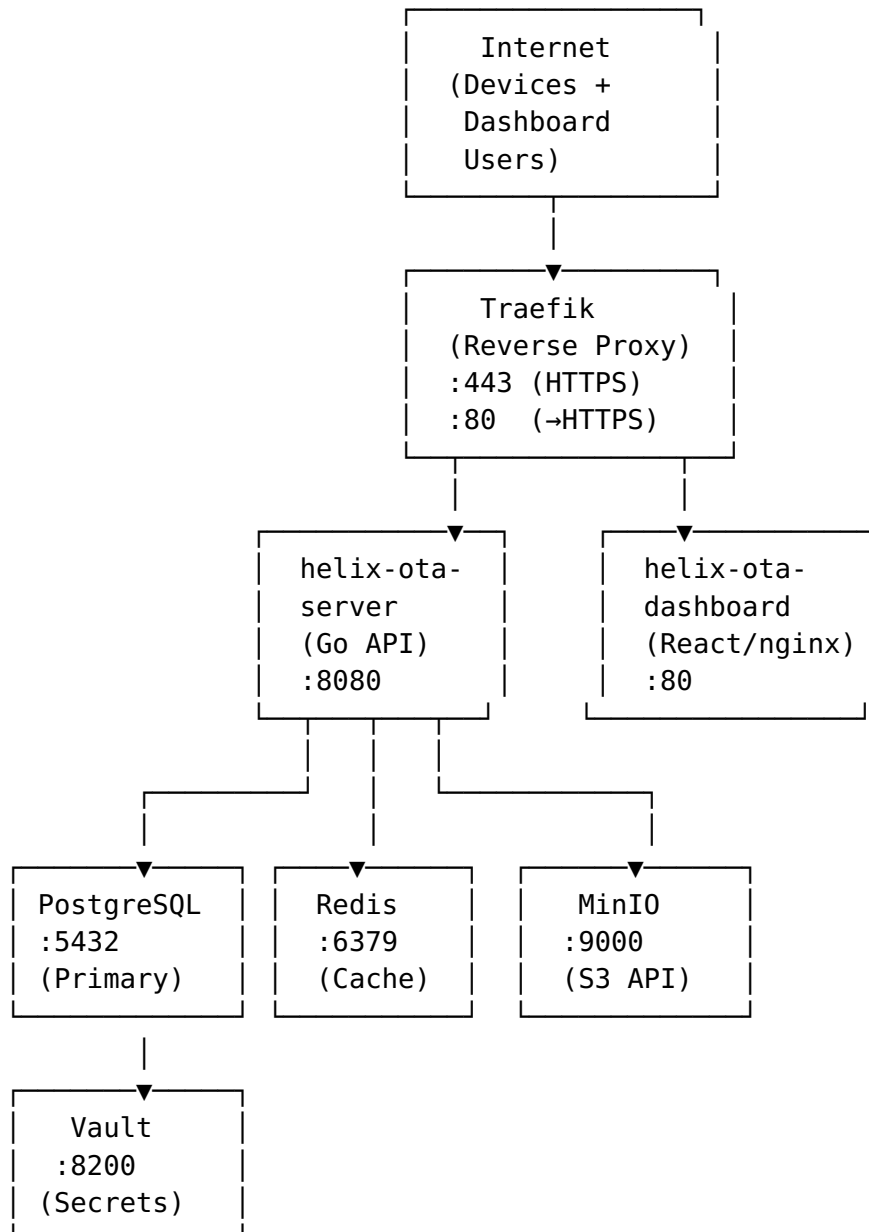
### 1.3 External Dependencies

| Dependency         | Purpose                                                                                | Required         |
|--------------------|----------------------------------------------------------------------------------------|------------------|
| <b>DNS</b>         | Service<br>discovery<br>within cluster;<br>external<br>domain<br>resolution            | Yes              |
| <b>SMTP Server</b> | Alert email<br>delivery<br>(security<br>notifications,<br>rollout alerts)              | Optional (1.0.0) |
| <b>CDN</b>         | Artifact<br>download<br>acceleration<br>for<br>geographically<br>distributed<br>fleets | Optional (1.0.0) |
| <b>HSM</b>         | Hardware-<br>backed signing<br>key storage<br>(production);                            | Recommended      |

| Dependency | Purpose                      | Required |
|------------|------------------------------|----------|
|            | Vault Transit<br>for non-HSM |          |

## 2. Container Architecture

### 2.1 Service Dependency Graph



## 2.2 Container Communication Matrix

All inter-service communication occurs over isolated Docker networks or Kubernetes network policies. No service exposes ports to the public internet except Traefik.

| Source                | Destination         | Protocol | Port | Authentication           | Purpose                                    |
|-----------------------|---------------------|----------|------|--------------------------|--------------------------------------------|
| Traefik               | helix-ota-server    | HTTP     | 8080 | None (internal)          | API request proxying                       |
| Traefik               | helix-ota-dashboard | HTTP     | 80   | None (internal)          | Dashboard static asset serving             |
| helix-ota-server      | PostgreSQL          | TCP      | 5432 | Password (pgxpool)       | Database queries                           |
| helix-ota-server      | Redis               | TCP      | 6379 | Password (if configured) | Cache read/write, rate limiting            |
| helix-ota-server      | MinIO               | HTTP     | 9000 | Access Key + Secret Key  | Artifact upload/download                   |
| helix-ota-server      | Vault               | HTTP     | 8200 | Vault Token / AppRole    | Signing key access, secret retrieval       |
| Devices (ext)         | Traefik             | HTTPS    | 443  | mTLS (device certs)      | Update check, artifact download, telemetry |
| Dashboard Users (ext) | Traefik             | HTTPS    | 443  | JWT Bearer               | Dashboard access, API calls                |
| MinIO Console (ext)   | Traefik             | HTTPS    | 9001 | MinIO credentials        | Admin console (optional)                   |

## 2.3 Container Security Hardening

Every container in the Helix OTA stack is hardened according to CIS Docker Benchmark and the security architecture defined in `SECURITY_ARCHITECTURE.md`:

| Hardening Measure           | Implementation                                                                |
|-----------------------------|-------------------------------------------------------------------------------|
| <b>Non-root user</b>        | All custom images run as UID 10001 (helixota);<br>USER helixota in Dockerfile |
| <b>Read-only filesystem</b> | read_only: true in compose; tmpfs mounts for writable paths                   |
| <b>No privileged mode</b>   | privileged: false enforced; no --cap-add ALL                                  |
| <b>Seccomp profile</b>      | Custom seccomp profile restricting syscalls to minimum required set           |
| <b>Resource limits</b>      | CPU and memory limits enforced on every container                             |
| <b>Image scanning</b>       | Trivy scans in CI pipeline; block deployment on CRITICAL/HIGH CVEs            |
| <b>Distroless base</b>      | Server image uses gcr.io/distroless/static-debian12 as final stage            |
| <b>No shell access</b>      | Distroless images contain no shell; prevents interactive container compromise |

## 3. Docker Compose Configuration

### 3.1 Development Docker Compose

The development docker-compose.yml is located at /containers/docker-compose.yml and orchestrates the complete stack locally. It is designed for developer workstation use with sensible defaults, hot-reload capabilities, and exposed debugging ports.

```
# containers/docker-compose.yml – Development Configuration
version: "3.9"
```

```
x-common-logging: &common-logging
  driver: json-file
  options:
    max-size: "10m"
```

```

    max-file: "3"

x-common-restart: &common-restart
  condition: on-failure
  max_attempts: 5

services:
  # =====
  # Traefik – Reverse Proxy / Ingress
  # =====
  traefik:
    image: traefik:v3.0
    container_name: helix-traefik
    command:
      - "--api.insecure=true" # Dev
        only: enable dashboard
      - "--providers.docker=true"
      - "--providers.docker.exposedbydefault=false"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.websecure.address=:443"
      - "--
        entrypoints.web.http.redirections.entryPoint.to=websecure"
      - "--
        entrypoints.web.http.redirections.entryPoint.scheme=https"
      - "--
        certificatesresolvers.letsencrypt.acme.tlschallenge=true"
      - "--certificatesresolvers.letsencrypt.acme.email=$
        {ACME_EMAIL:-admin@helix-ota.io}"
      - "--certificatesresolvers.letsencrypt.acme.storage=/
        letsencrypt/acme.json"
      - "--metrics.prometheus=true"
      - "--metrics.prometheus.addEntryPointsLabels=true"
      - "--metrics.prometheus.addServicesLabels=true"
      - "--accesslog=true"
      - "--accesslog.format=json"
    ports:
      - "80:80"
      - "443:443"
      - "8080:8080" # Traefik dashboard (dev only)
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - traefik-letsencrypt:/letsencrypt
    networks:
      - helix-frontend
      - helix-backend
  restart: *common-restart
  logging: *common-logging
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.traefik.rule=Host(`traefik.${
      DOMAIN:-localhost}`)"
    - "traefik.http.routers.traefik.entrypoints=websecure"
    - "traefik.http.routers.traefik.tls=true"

```

```

- "traefik.http.routers.traefik.tls.certresolver=letsencrypt"
- "traefik.http.routers.traefik.service=api@internal"

# =====
# Helix OTA Server – Go REST API
# =====
helix-ota-server:
  build:
    context: ../server
    dockerfile: Dockerfile
    target:
      development # Uses builder stage with Delve
                  debugger
  container_name: helix-ota-server
  environment:
    HELIX_DATABASE_URL: "postgres://helix_ota:${PG_PASSWORD:-helix_ota_dev}@postgresql:5432/helix_ota?
                        sslmode=disable&pool_max_conns=50&pool_min_conns=10"
    HELIX_REDIS_URL: "redis://:${REDIS_PASSWORD:-helix_redis_dev}@redis:6379/0"
    HELIX_MINIO_ENDPOINT: "minio:9000"
    HELIX_MINIO_ACCESS_KEY: "${MINIO_ACCESS_KEY:-minioadmin}"
    HELIX_MINIO_SECRET_KEY: "${MINIO_SECRET_KEY:-minioadmin}"
    HELIX_MINIO_BUCKET: "helix-ota-artifacts"
    HELIX_MINIO_USE_SSL: "false"
    HELIX_VAULT_ADDRESS: "http://vault:8200"
    HELIX_VAULT_TOKEN: "${VAULT_DEV_TOKEN:-dev-only-token}"
    HELIX_VAULT_TRANSIT_KEY: "artifact-signing-key"
    HELIX_JWT_SIGNING_KEY: "${JWT_SIGNING_KEY:-dev-only-jwt-key-change-in-prod}"
    HELIX_JWT_ACCESS_TTL: "15m"
    HELIX_JWT_REFRESH_TTL: "168h" # 7 days
    HELIX_SERVER_PORT: "8080"
    HELIX_SERVER_READ_TIMEOUT: "30s"
    HELIX_SERVER_WRITE_TIMEOUT: "60s"
    HELIX_SERVER_IDLE_TIMEOUT: "120s"
    HELIX_LOG_LEVEL: "${LOG_LEVEL:-debug}"
    HELIX_LOG_FORMAT: "json"
    HELIX_ARTIFACT_MAX_SIZE: "2147483648" # 2 GiB
    HELIX_ROLLOUT_AUTO_ADVANCE_INTERVAL: "15m"
    HELIX_ROLLOUT_HEALTH_CHECK_INTERVAL: "5m"
    HELIX_RATE_LIMIT_DEVICE_RPM: "60"
    HELIX_RATE_LIMIT_DASHBOARD_RPM: "120"
    HELIX_RATE_LIMIT_LOGIN_RPM: "10"
    HELIX_METRICS_ENABLED: "true"
    HELIX_METRICS_PATH: "/metrics"
    HELIX_TRACING_ENABLED: "true"
    HELIX_TRACING_ENDPOINT: "http://otel-collector:4318"
  ports:

```

```

- "9090:8080"      # API server (mapped to 9090 to
                    # avoid Traefik conflict)
- "2345:2345"      # Delve debugger (dev only)
volumes:
- ../server:/app    # Hot-reload source mount
- server-tmp:/tmp    # Temp storage for artifact
                    # uploads
depends_on:
  postgresql:
    condition: service_healthy
  redis:
    condition: service_healthy
  minio:
    condition: service_healthy
  vault:
    condition: service_started
networks:
- helix-backend
restart: *common-restart
logging: *common-logging
healthcheck:
  test: ["CMD", "wget", "--spider", "-q", "http://
        localhost:8080/healthz"]
  interval: 10s
  timeout: 5s
  retries: 3
  start_period: 30s
labels:
- "traefik.enable=true"
- "traefik.http.routers.api.rule=Host(`${DOMAIN:-
  api.helix-ota.io}`)"
- "traefik.http.routers.api.entrypoints=websecure"
- "traefik.http.routers.api.tls=true"
- "traefik.http.routers.api.tls.certresolver=letsencrypt"
- "traefik.http.services.api.loadbalancer.server.port=8080"

# =====
# Helix OTA Dashboard – React SPA + nginx
# =====
helix-ota-dashboard:
  build:
    context: ../dashboard
    dockerfile: Dockerfile
    target: production
  container_name: helix-ota-dashboard
  environment:
    VITE_API_BASE_URL: "${API_BASE_URL:-https://api.helix-
      ota.io}"
    VITE_WS_URL: "${WS_URL:-wss://api.helix-ota.io/ws}"
  ports:
- "3000:80"      # Dev: direct access bypassing Traefik
  networks:
- helix-frontend

```

```

    - helix-backend
restart: *common-restart
logging: *common-logging
healthcheck:
  test: ["CMD", "wget", "--spider", "-q", "http://localhost:80/healthz"]
  interval: 30s
  timeout: 5s
  retries: 3
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.dashboard.rule=Host(`${DOMAIN:-app.helix-ota.io}`)"
  - "traefik.http.routers.dashboard.entrypoints=websecure"
  - "traefik.http.routers.dashboard.tls=true"
  - "traefik.http.routers.dashboard.tls.certresolver=letsencrypt"
  - "traefik.http.services.dashboard.loadbalancer.server.port=80"

# =====
# PostgreSQL 16 – Primary Database
# =====
postgresql:
  image: postgres:16-alpine
  container_name: helix-postgresql
  environment:
    POSTGRES_DB: "helix_ota"
    POSTGRES_USER: "helix_ota"
    POSTGRES_PASSWORD: "${PG_PASSWORD:-helix_ota_dev}"
    PGDATA: "/var/lib/postgresql/data/pgdata"
  command:
    - "postgres"
    - "-c", "max_connections=200"
    - "-c", "shared_buffers=512MB"
    - "-c", "effective_cache_size=1536MB"
    - "-c", "work_mem=10MB"
    - "-c", "maintenance_work_mem=256MB"
    - "-c", "wal_level=replica"
    - "-c", "max_wal_size=2GB"
    - "-c", "min_wal_size=512MB"
    - "-c", "checkpoint_completion_target=0.9"
    - "-c", "random_page_cost=1.1" # SSD-optimized
    - "-c", "log_min_duration_statement=200" # Log slow queries > 200ms
    - "-c", "log_checkpoints=on"
    - "-c", "log_connections=on"
    - "-c", "log_disconnections=on"
    - "-c", "ssl=on"
    - "-c", "ssl_cert_file=/etc/postgresql/tls/server.crt"
    - "-c", "ssl_key_file=/etc/postgresql/tls/server.key"
  ports:
    - "5432:5432" # Dev: direct DB access

```

```

volumes:
  - postgres-data:/var/lib/postgresql/data
  - ./init-db:/docker-entrypoint-initdb.d:ro
networks:
  - helix-backend
restart: always
logging: *common-logging
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U helix_ota -d
    helix_ota"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 30s
shm_size: "256mb"

# =====
# Redis 7 – Cache & Session Store
# =====

redis:
  image: redis:7-alpine
  container_name: helix-redis
  command: >
    redis-server
    --requirepass ${REDIS_PASSWORD:-helix_redis_dev}
    --maxmemory 384mb
    --maxmemory-policy allkeys-lru
    --appendonly yes
    --appendfsync everysec
    --save 60 1000
    --save 300 100
    --save 900 1
    --tcp-backlog 511
    --timeout 300
    --tcp-keepalive 60
    --hz 10
    --dynamic-hz yes
  ports:
    - "6379:6379"    # Dev: direct Redis access
  volumes:
    - redis-data:/data
  networks:
    - helix-backend
  restart: always
  logging: *common-logging
  healthcheck:
    test: ["CMD", "redis-cli", "-a", "${REDIS_PASSWORD:-
      helix_redis_dev}", "ping"]
    interval: 10s
    timeout: 5s
    retries: 5

```

```

# =====
# MinIO – S3-Compatible Artifact Storage
# =====
minio:
  image: minio/minio:latest
  container_name: helix-minio
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: "${MINIO_ACCESS_KEY:-minioadmin}"
    MINIO_ROOT_PASSWORD: "${MINIO_SECRET_KEY:-minioadmin}"
    MINIO_BROWSER_REDIRECT_URL: "https://minio.${DOMAIN:-localhost}"
    MINIO_SERVER_URL: "https://s3.${DOMAIN:-localhost}"
    MINIO_REGION: "${MINIO_REGION:-us-east-1}"
  ports:
    - "9000:9000"      # S3 API
    - "9001:9001"      # Console
  volumes:
    - minio-data:/data
  networks:
    - helix-backend
  restart: always
  logging: *common-logging
  healthcheck:
    test: ["CMD", "mc", "ready", "local"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 30s

# =====
# HashiCorp Vault – Secret Management
# =====
vault:
  image: hashicorp/vault:1.15
  container_name: helix-vault
  command: server -dev -dev-root-token-id="$
    {VAULT_DEV_TOKEN:-dev-only-token}" -dev-listen-
    address=0.0.0.0:8200
  environment:
    VAULT_DEV_ROOT_TOKEN_ID: "${VAULT_DEV_TOKEN:-dev-only-
    token}"
    VAULT_DEV_LISTEN_ADDRESS: "0.0.0.0:8200"
    VAULT_API_ADDR: "http://vault:8200"
  ports:
    - "8200:8200"      # Dev: direct Vault access
  volumes:
    - vault-data:/vault/data
    - ./vault-config:/vault/config:ro
  networks:
    - helix-backend
  restart: *common-restart

```

```

logging: *common-logging
healthcheck:
  test: ["CMD", "vault", "status"]
  interval: 10s
  timeout: 5s
  retries: 3
cap_add:
  - IPC_LOCK      # Required for mlock

# =====
# MinIO Initialization – Bucket Creation
# =====
minio-init:
  image: minio/mc:latest
  container_name: helix-minio-init
  depends_on:
    minio:
      condition: service_healthy
  entrypoint: >
    /bin/sh -c "
    mc alias set helix-minio http://minio:9000 $
      {MINIO_ACCESS_KEY:-minioadmin} ${MINIO_SECRET_KEY:-
        minioadmin};
    mc mb --ignore-existing helix-minio/helix-ota-
      artifacts;
    mc anonymous set download helix-minio/helix-ota-
      artifacts;
    echo 'MinIO bucket initialized successfully';
    "

  networks:
    - helix-backend

# =====
# Vault Initialization – Secrets Engine Setup
# =====
vault-init:
  image: hashicorp/vault:1.15
  container_name: helix-vault-init
  depends_on:
    vault:
      condition: service_started
  entrypoint: >
    /bin/sh -c "
    export VAULT_ADDR='http://vault:8200';
    vault login ${VAULT_DEV_TOKEN:-dev-only-token};
    vault secrets enable -path=transit transit 2>/dev/null
      || true;
    vault write -f transit/keys/artifact-signing-key
      type=rsa-4096 2>/dev/null || true;
    vault secrets enable -path=secret kv 2>/dev/null ||
      true;

```

```

vault kv put secret/helix-ota jwt-signing-key='$
  {JWT_SIGNING_KEY:-dev-only-jwt-key-change-in-
  prod}';
vault kv put secret/helix-ota totp-encryption-key='$
  (openssl rand -base64 32)';
vault kv put secret/helix-ota/db-credentials
  username='helix_ota' password='${PG_PASSWORD:-
  helix_ota_dev}';
echo 'Vault initialized successfully';
"

```

```

networks:
  - helix-backend

```

```

# =====

```

```

# Networks

```

```

# =====

```

```

networks:
  helix-frontend:
    driver: bridge
    name: helix-frontend
    ipam:
      config:
        - subnet: 172.28.0.0/16
  helix-backend:
    driver: bridge
    name: helix-backend
    internal: true # No external routing
    ipam:
      config:
        - subnet: 172.29.0.0/16

```

```

# =====

```

```

# Volumes

```

```

# =====

```

```

volumes:
  postgres-data:
    driver: local
    name: helix-postgres-data
  redis-data:
    driver: local
    name: helix-redis-data
  minio-data:
    driver: local
    name: helix-minio-data
  vault-data:
    driver: local
    name: helix-vault-data
  traefik-letsencrypt:
    driver: local
    name: helix-traefik-letsencrypt
  server-tmp:

```

```
driver: local
name: helix-server-tmp
```

## 3.2 Production Overrides

For production, use `docker-compose.prod.yml` overrides:

```
# containers/docker-compose.prod.yml
version: "3.9"

services:
  traefik:
    command:
      - "--api.insecure=false"      # Disable dashboard
      - "--providers.docker=true"
      - "--providers.docker.exposedbydefault=false"
      - "--entrypoints.websecure.address=:443"
      - "--
        certificatesresolvers.letsencrypt.acme.tlschallenge=true"
      - "--certificatesresolvers.letsencrypt.acme.email=${
        {ACME_EMAIL}"
      - "--certificatesresolvers.letsencrypt.acme.storage=/
        letsencrypt/acme.json"
      - "--metrics.prometheus=true"
      - "--accesslog=true"
      - "--accesslog.format=json"
    deploy:
      replicas: 2
      resources:
        limits:
          cpus: "1.0"
          memory: 512M

  helix-ota-server:
    build:
      target: production
    environment:
      HELIX_LOG_LEVEL: "info"
      HELIX_VAULT_TOKEN: ""          # Use AppRole instead
      HELIX_VAULT_ROLE_ID: "${VAULT_ROLE_ID}"
      HELIX_VAULT_SECRET_ID: "${VAULT_SECRET_ID}"
    volumes: []                     # No source mount in
      prod
    deploy:
      replicas: 3
      resources:
        limits:
          cpus: "2.0"
          memory: 1G
        reservations:
          cpus: "0.5"
          memory: 256M
```

```

postgresql:
  environment:
    POSTGRES_PASSWORD: "${PG_PASSWORD}"    # No default in
    prod
  command:
    - "postgres"
    - "-c", "max_connections=200"
    - "-c", "shared_buffers=2GB"
    - "-c", "effective_cache_size=6GB"
    - "-c", "work_mem=20MB"
    - "-c", "maintenance_work_mem=1GB"
    - "-c", "wal_level=replica"
    - "-c", "archive_mode=on"
    - "-c", "archive_command=cp %p /var/lib/postgresql/
      wal_archive/%f"
    - "-c", "max_wal_size=4GB"
  deploy:
    resources:
      limits:
        cpus: "4.0"
        memory: 8G

```

```

redis:
  command: >
    redis-server
    --requirepass ${REDIS_PASSWORD}
    --maxmemory 768mb
    --maxmemory-policy allkeys-lru
    --appendonly yes
    --appendfsync everysec
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: 1G

```

```

vault:
  command: server -config=/vault/config/production.hcl
  environment:
    VAULT_API_ADDR: "https://vault.${DOMAIN}:8200"
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: 512M

```

```

minio:
  environment:
    MINIO_ROOT_USER: "${MINIO_ACCESS_KEY}"
    MINIO_ROOT_PASSWORD: "${MINIO_SECRET_KEY}"
  deploy:

```

```
resources:
  limits:
    cpus: "2.0"
    memory: 2G
```

---

## 4. Kubernetes Deployment

### 4.1 Helm Chart Structure

```
helm/helix-ota/
├── Chart.yaml
├── values.yaml
├── values-production.yaml
├── templates/
│   ├── _helpers.tpl
│   ├── server-deployment.yaml
│   ├── server-service.yaml
│   ├── server-hpa.yaml
│   ├── server-pdb.yaml
│   ├── dashboard-deployment.yaml
│   ├── dashboard-service.yaml
│   ├── postgresql-statefulset.yaml
│   ├── postgresql-service.yaml
│   ├── redis-statefulset.yaml
│   ├── redis-service.yaml
│   ├── minio-statefulset.yaml
│   ├── minio-service.yaml
│   ├── vault-statefulset.yaml
│   ├── vault-service.yaml
│   ├── ingress.yaml
│   ├── networkpolicy.yaml
│   ├── secrets.yaml
│   ├── configmap.yaml
│   ├── serviceaccount.yaml
│   └── prometheus-servicemonitor.yaml
```

### 4.2 Helm Values (Complete)

```
# helm/helix-ota/values.yaml
global:
  domain: "helix-ota.io"
  environment: "production"
  imagePullSecrets: []
  storageClass: "ssd"

# -- Server Configuration
server:
  replicaCount: 3
```

```
image:
  repository: ghcr.io/vasic-digital/helix-ota-server
  tag: "1.0.0"
  pullPolicy: IfNotPresent
resources:
  requests:
    cpu: 500m
    memory: 256Mi
  limits:
    cpu: "2"
    memory: 1Gi
autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 20
  targetCPUUtilizationPercentage: 70
  targetMemoryUtilizationPercentage: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
          value: 25
          periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Percent
          value: 100
          periodSeconds: 30
podDisruptionBudget:
  minAvailable: 2
service:
  type: ClusterIP
  port: 8080
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 15
  periodSeconds: 20
  timeoutSeconds: 5
  failureThreshold: 3
readinessProbe:
  httpGet:
    path: /readyz
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  timeoutSeconds: 3
  failureThreshold: 3
env:
```

```

    logLevel: "info"
    logFormat: "json"
    artifactMaxSize: "2147483648"
    rolloutAutoAdvanceInterval: "15m"
    rolloutHealthCheckInterval: "5m"
    rateLimitDeviceRPM: "60"
    rateLimitDashboardRPM: "120"
    rateLimitLoginRPM: "10"
    metricsEnabled: "true"
    metricsPath: "/metrics"
    tracingEnabled: "true"
secrets:
  jwtSigningKey: ""          # Set via --set or external
                             secret manager
  vaultRoleID: ""
  vaultSecretID: ""

# -- Dashboard Configuration
dashboard:
  replicaCount: 2
  image:
    repository: ghcr.io/vasic-digital/helix-ota-dashboard
    tag: "1.0.0"
    pullPolicy: IfNotPresent
  resources:
    requests:
      cpu: 100m
      memory: 64Mi
    limits:
      cpu: 250m
      memory: 128Mi
  service:
    type: ClusterIP
    port: 80

# -- PostgreSQL Configuration
postgresql:
  enabled: true
  image:
    repository: postgres
    tag: "16-alpine"
  auth:
    database: "helix_ota"
    username: "helix_ota"
    existingSecret: "helix-ota-postgresql"
  primary:
    resources:
      requests:
        cpu: "1"
        memory: 2Gi
      limits:
        cpu: "4"

```

```
        memory: 8Gi
persistence:
  enabled: true
  size: 100Gi
  storageClass: "ssd"
podSecurityContext:
  fsGroup: 999
containerSecurityContext:
  runAsNonRoot: true
  runAsUser: 999
readReplicas:
  enabled: true
  replicaCount: 2
resources:
  requests:
    cpu: "1"
    memory: 2Gi
  limits:
    cpu: "2"
    memory: 4Gi

# -- Redis Configuration
redis:
  enabled: true
  image:
    repository: redis
    tag: "7-alpine"
  auth:
    existingSecret: "helix-ota-redis"
  master:
    resources:
      requests:
        cpu: 250m
        memory: 256Mi
      limits:
        cpu: "1"
        memory: 1Gi
    persistence:
      enabled: true
      size: 5Gi
  replica:
    replicaCount: 2
    resources:
      requests:
        cpu: 250m
        memory: 256Mi
      limits:
        cpu: 500m
        memory: 512Mi

# -- MinIO Configuration
minio:
```

```
enabled: true
image:
  repository: minio/minio
  tag: "latest"
mode: distributed
auth:
  existingSecret: "helix-ota-minio"
buckets:
  - name: helix-ota-artifacts
    policy: none
    purge: false
resources:
  requests:
    cpu: 500m
    memory: 512Mi
  limits:
    cpu: "2"
    memory: 2Gi
persistence:
  enabled: true
  size: 500Gi
  storageClass: "ssd"

# -- Vault Configuration
vault:
  enabled: true
  image:
    repository: hashicorp/vault
    tag: "1.15"
  mode: standalone # Use ha for production with Raft
  auth:
    existingSecret: "helix-ota-vault-unseal"
  resources:
    requests:
      cpu: 250m
      memory: 256Mi
    limits:
      cpu: "1"
      memory: 512Mi
  persistence:
    enabled: true
    size: 10Gi
  injector:
    enabled: true

# -- Ingress Configuration
ingress:
  enabled: true
  className: "traefik"
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
```

```

    traefik.ingress.kubernetes.io/router.entrypoints:
      "websecure"
    traefik.ingress.kubernetes.io/router.tls: "true"
  tls: true
  apiHost: "api.helix-ota.io"
  dashboardHost: "app.helix-ota.io"

# -- Observability
observability:
  serviceMonitor:
    enabled: true
    interval: 15s
    path: /metrics
    labels:
      release: prometheus
  grafana:
    dashboardsEnabled: true

```

## 4.3 Server Deployment Manifest

```

# templates/server-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "helix-ota.fullname" . }}-server
  labels:
    {{- include "helix-ota.labels" . | nindent 4 }}
    app.kubernetes.io/component: server
spec:
  replicas: {{ .Values.server.replicaCount }}
  selector:
    matchLabels:
      {{- include "helix-ota.selectorLabels" . | nindent 6 }}
      app.kubernetes.io/component: server
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print
          $.Template.BasePath "/configmap.yaml") . |
          sha256sum }}
        checksum/secrets: {{ include (print
          $.Template.BasePath "/secrets.yaml") . |
          sha256sum }}
        prometheus.io/scrape: "true"
        prometheus.io/port: "8080"
        prometheus.io/path: "/metrics"
      labels:
        {{- include "helix-ota.selectorLabels" . | nindent 8 }}
        app.kubernetes.io/component: server
    spec:

```

```

serviceAccountName: {{ include "helix-
  ota.serviceAccountName" . }}
securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  runAsGroup: 10001
  fsGroup: 10001
  seccompProfile:
    type: RuntimeDefault
containers:
  - name: server
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
    image: "{{ .Values.server.image.repository }}:
{{ .Values.server.image.tag }}"
    imagePullPolicy:
{{ .Values.server.image.pullPolicy }}
    ports:
      - name: http
        containerPort: 8080
        protocol: TCP
      - name: metrics
        containerPort: 8080
        protocol: TCP
    envFrom:
      - configMapRef:
          name: {{ include "helix-ota.fullname" . }}-
server-config
      - secretRef:
          name: {{ include "helix-ota.fullname" . }}-
server-secrets
    env:
      - name: HELIX_DATABASE_URL
        valueFrom:
          secretKeyRef:
            name:
{{ .Values.postgresql.auth.existingSecret }}
            key: uri
      - name: HELIX_REDIS_URL
        valueFrom:
          secretKeyRef:
            name:
{{ .Values.redis.auth.existingSecret }}
            key: uri
      - name: HELIX_VAULT_TOKEN
        valueFrom:
          secretKeyRef:
            name: {{ include "helix-
ota.fullname" . }}-server-secrets
            key: vault-token

```

```

    volumeMounts:
      - name: tmp
        mountPath: /tmp
      - name: tls-certs
        mountPath: /etc/helix-ota/tls
        readOnly: true
    livenessProbe: {{-
toYaml .Values.server.livenessProbe | nindent 12 }}
    readinessProbe: {{-
toYaml .Values.server.readinessProbe | nindent
12 }}
    resources: {{- toYaml .Values.server.resources |
nindent 12 }}
  volumes:
    - name: tmp
      emptyDir:
        medium: Memory
        sizeLimit: 256Mi
    - name: tls-certs
      secret:
        secretName: {{ include "helix-
ota.fullname" . }}-server-tls
        optional: true
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector:
        matchLabels:
          app.kubernetes.io/component: server

```

## 4.4 Horizontal Pod Autoscaler

```

# templates/server-hpa.yaml
{{- if .Values.server.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ include "helix-ota.fullname" . }}-server-hpa
  labels:
    {{- include "helix-ota.labels" . | nindent 4 }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ include "helix-ota.fullname" . }}-server
  minReplicas: {{ .Values.server.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.server.autoscaling.maxReplicas }}
  metrics:
    - type: Resource
      resource:
        name: cpu

```

```

        target:
          type: Utilization
          averageUtilization:
            {{ .Values.server.autoscaling.targetCPUUtilizationPercentage }}
- type: Resource
  resource:
    name: memory
    target:
      type: Utilization
      averageUtilization:
        {{ .Values.server.autoscaling.targetMemoryUtilizationPercentage }}
behavior:
  scaleDown:
    stabilizationWindowSeconds: 300
    policies:
      - type: Percent
        value: 25
        periodSeconds: 60
  scaleUp:
    stabilizationWindowSeconds: 60
    policies:
      - type: Percent
        value: 100
        periodSeconds: 30
{{- end }}

```

## 4.5 Network Policies

```

# templates/networkpolicy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: {{ include "helix-ota.fullname" . }}-network-policy
  namespace: {{ .Release.Namespace }}
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/part-of: {{ include "helix-ota.name" . }}
  policyTypes:
    - Ingress
    - Egress
  ingress:
    # Traefik → Server
    - from:
        - podSelector:
            matchLabels:
              app.kubernetes.io/name: traefik
      ports:
        - port: 8080
          protocol: TCP
    # Traefik → Dashboard

```

```

- from:
  - podSelector:
      matchLabels:
        app.kubernetes.io/name: traefik
  ports:
    - port: 80
      protocol: TCP
# Prometheus → Server metrics
- from:
  - namespaceSelector:
      matchLabels:
        name: monitoring
  - podSelector:
      matchLabels:
        app.kubernetes.io/name: prometheus
  ports:
    - port: 8080
      protocol: TCP
egress:
# Server → PostgreSQL
- to:
  - podSelector:
      matchLabels:
        app.kubernetes.io/component: postgresql
  ports:
    - port: 5432
      protocol: TCP
# Server → Redis
- to:
  - podSelector:
      matchLabels:
        app.kubernetes.io/component: redis
  ports:
    - port: 6379
      protocol: TCP
# Server → MinIO
- to:
  - podSelector:
      matchLabels:
        app.kubernetes.io/component: minio
  ports:
    - port: 9000
      protocol: TCP
# Server → Vault
- to:
  - podSelector:
      matchLabels:
        app.kubernetes.io/component: vault
  ports:
    - port: 8200
      protocol: TCP
# DNS resolution

```

```
- to: []
ports:
  - port: 53
    protocol: UDP
  - port: 53
    protocol: TCP
```

---

## 5. Environment Configuration

### 5.1 Complete Environment Variable Reference

The Helix OTA server is configured entirely through environment variables, loaded by the `vasic-digital/config` submodule. All variables are prefixed with `HELIX_` to avoid collisions.

#### Server Configuration

| Variable                               | Type     | Default | Required | Description                                               |
|----------------------------------------|----------|---------|----------|-----------------------------------------------------------|
| HELIX_SERVER_PORT                      | int      | 8080    | No       | HTTP port                                                 |
| HELIX_SERVER_READ_TIMEOUT              | duration | 30s     | No       | Maximum duration for reading the entire request           |
| HELIX_SERVER_WRITE_TIMEOUT             | duration | 60s     | No       | Maximum duration before timing out write of the response  |
| HELIX_SERVER_IDLE_TIMEOUT              | duration | 120s    | No       | Maximum amount of time for the request to be idle         |
| HELIX_SERVER_GRACEFUL_SHUTDOWN_TIMEOUT | duration | 30s     | No       | Maximum time for the connection to finish during shutdown |

---

Database Configuration

| Variable                          | Type     | Default | Required | Description                                                                                  |
|-----------------------------------|----------|---------|----------|----------------------------------------------------------------------------------------------|
| HELIX_DATABASE_URL                | string   | —       | Yes      | PostgreSQL (format): postgres://username:password@hostname:port/db?sslmode=full&pool_size=10 |
| HELIX_DATABASE_MAX_OPEN_CONNS     | int      | 50      | No       | Maximum number of open database connections (production)                                     |
| HELIX_DATABASE_MAX_IDLE_CONNS     | int      | 10      | No       | Minimum number of idle database connections                                                  |
| HELIX_DATABASE_CONN_MAX_IDLE_TIME | duration | 15m     | No       | Maximum idle time for database connections (收)                                               |
| HELIX_DATABASE_CONN_MAX_LIFETIME  | duration | 30m     | No       | Maximum lifetime for database connections                                                    |

Redis Configuration

| Variable                      | Type     | Default          | Required | Description                          |
|-------------------------------|----------|------------------|----------|--------------------------------------|
| HELIX_REDIS_URL               | string   | —                | Yes      | Redis URL (redis://hostname:port/db) |
| HELIX_REDIS_CACHE_TTL         | duration | 60s              | No       | Default cache TTL (device health)    |
| HELIX_REDIS_RATE_LIMIT_PREFIX | string   | helix:ratelimit: | No       | Key prefix for rate limit counter    |
| HELIX_REDIS_SESSION_PREFIX    | string   | helix:session:   | No       | Key prefix for session               |

MinIO / S3 Configuration

| Variable               | Type   | Default             | Required | Description                                              |
|------------------------|--------|---------------------|----------|----------------------------------------------------------|
| HELIX_MINIO_ENDPOINT   | string | —                   | Yes      | MinIO/S3 endpoint (e.g., minio:9000 or s3.amazonaws.com) |
| HELIX_MINIO_ACCESS_KEY | string | —                   | Yes      | Access key for S3/MinIO authentication                   |
| HELIX_MINIO_SECRET_KEY | string | —                   | Yes      | Secret key for S3/MinIO authentication                   |
| HELIX_MINIO_BUCKET     | string | helix-ota-artifacts | No       | Bucket name for artifact storage                         |
| HELIX_MINIO_REGION     | string | us-east-1           | No       | S3 region                                                |

| Variable            | Type | Default | Required | Description                                                |
|---------------------|------|---------|----------|------------------------------------------------------------|
| HELIX_MINIO_USE_SSL | bool | true    | No       | Enable SSL for S3 connections (must be true in production) |

### Vault Configuration

| Variable                  | Type   | Default                              | Required    | Description                                       |
|---------------------------|--------|--------------------------------------|-------------|---------------------------------------------------|
| HELIX_VAULT_ADDRESS       | string | —                                    | Yes         | Vault server address (e.g., http://vault:8200)    |
| HELIX_VAULT_TOKEN         | string | —                                    | Conditional | Vault token (dev only; use AppRole in production) |
| HELIX_VAULT_ROLE_ID       | string | —                                    | Conditional | Vault AppRole Role ID (production)                |
| HELIX_VAULT_SECRET_ID     | string | —                                    | Conditional | Vault AppRole Secret ID (production)              |
| HELIX_VAULT_TRANSIT_KEY   | string | artifact-signing-key                 | No          | Transit engine key name for artifact signing      |
| HELIX_VAULT_TOTP_KEY_PATH | string | secret/helix-ota/totp-encryption-key | No          | KV path for TOTP secret encryption key            |

### JWT / Authentication Configuration

| Variable              | Type   | Default | Required | Description                                                 |
|-----------------------|--------|---------|----------|-------------------------------------------------------------|
| HELIX_JWT_SIGNING_KEY | string | —       | Yes      | RSA private key PEM for JWT signing (or reference to Vault) |

| Variable                | Type     | Default                   | Required | Description                                         |
|-------------------------|----------|---------------------------|----------|-----------------------------------------------------|
| HELIX_JWT_ACCESS_TTL    | duration | 15m                       | No       | Access token time-to-live                           |
| HELIX_JWT_REFRESH_TTL   | duration | 168h                      | No       | Refresh token time-to-live (7 days)                 |
| HELIX_JWT_ISSUER        | string   | helix-ota                 | No       | JWT issuer claim                                    |
| HELIX_MTLS_CA_CERT_PATH | string   | /etc/helix-ota/tls/ca.crt | No       | Path to device CA certificate for mTLS verification |
| HELIX_MTLS_CRL_PATH     | string   | /etc/helix-ota/tls/ca.crl | No       | Path to Certificate Revocation List                 |

### Rollout Configuration

| Variable                              | Type     | Default | Required | Description                                      |
|---------------------------------------|----------|---------|----------|--------------------------------------------------|
| HELIX_ROLLOUT_AUTO_ADVANCE_INTERVAL   | duration | 15m     | No       | Interval for auto-advance evaluation job         |
| HELIX_ROLLOUT_HEALTH_CHECK_INTERVAL   | duration | 5m      | No       | Interval for rollout health metric aggregation   |
| HELIX_ROLLOUT_AUTO_ROLLBACK_THRESHOLD | float    | 0.05    | No       | Failure threshold (5%) to trigger auto-rollback  |
| HELIX_ROLLOUT_DEFAULT_INCREMENT_STEP  | int      | 10      | No       | Default percentage increment for gradual rollout |

Artifact Configuration

| Variable                          | Type   | Default    | Required | Description                                     |
|-----------------------------------|--------|------------|----------|-------------------------------------------------|
| HELIX_ARTIFACT_MAX_SIZE           | int64  | 2147483648 | No       | Maximum artifact upload size in bytes (2 GiB)   |
| HELIX_ARTIFACT_TEMP_DIR           | string | /tmp       | No       | Temporary directory for upload staging          |
| HELIX_ARTIFACT_VALIDATION_WORKERS | int    | 4          | No       | Number of goroutine pool workers for validation |

Rate Limiting Configuration

| Variable                             | Type | Default | Required | Description                            |
|--------------------------------------|------|---------|----------|----------------------------------------|
| HELIX_RATE_LIMIT_DEVICE_RPM          | int  | 60      | No       | Requests per minute per device         |
| HELIX_RATE_LIMIT_DASHBOARD_RPM       | int  | 120     | No       | Requests per minute per dashboard user |
| HELIX_RATE_LIMIT_LOGIN_RPM           | int  | 10      | No       | Login attempts per minute per IP       |
| HELIX_RATE_LIMIT_ARTIFACT_UPLOAD_RPH | int  | 10      | No       | Artifact uploads per hour per user     |

Observability Configuration

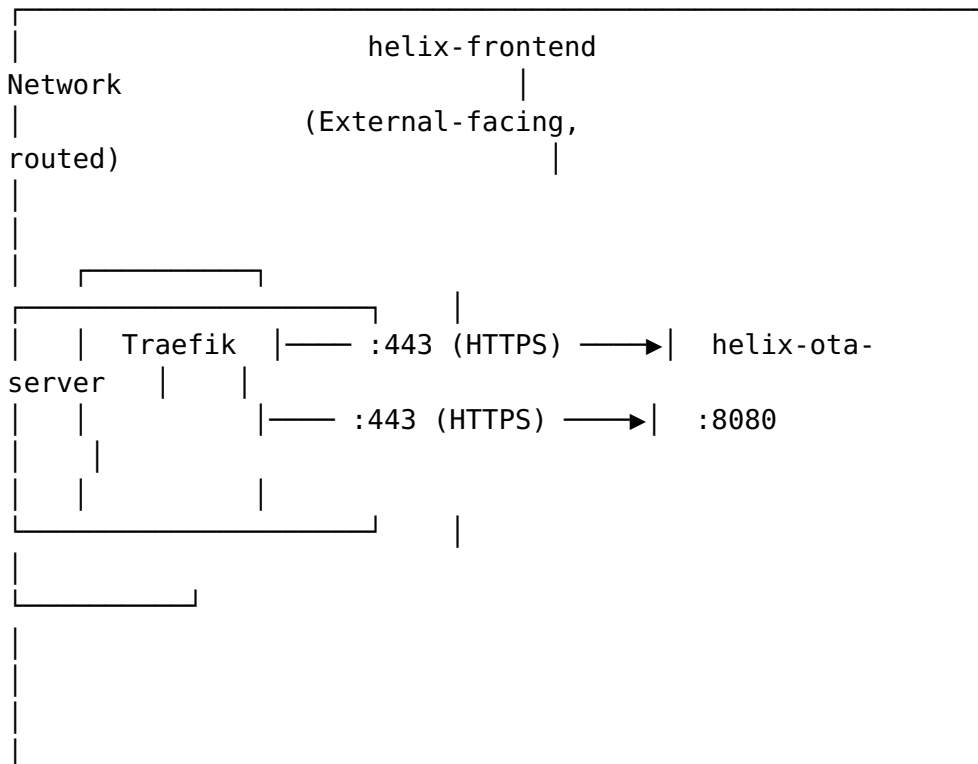
| Variable         | Type   | Default | Required | Description                         |
|------------------|--------|---------|----------|-------------------------------------|
| HELIX_LOG_LEVEL  | string | info    | No       | Log level: debug, info, warn, error |
| HELIX_LOG_FORMAT | string | json    | No       | Log format: json, text              |

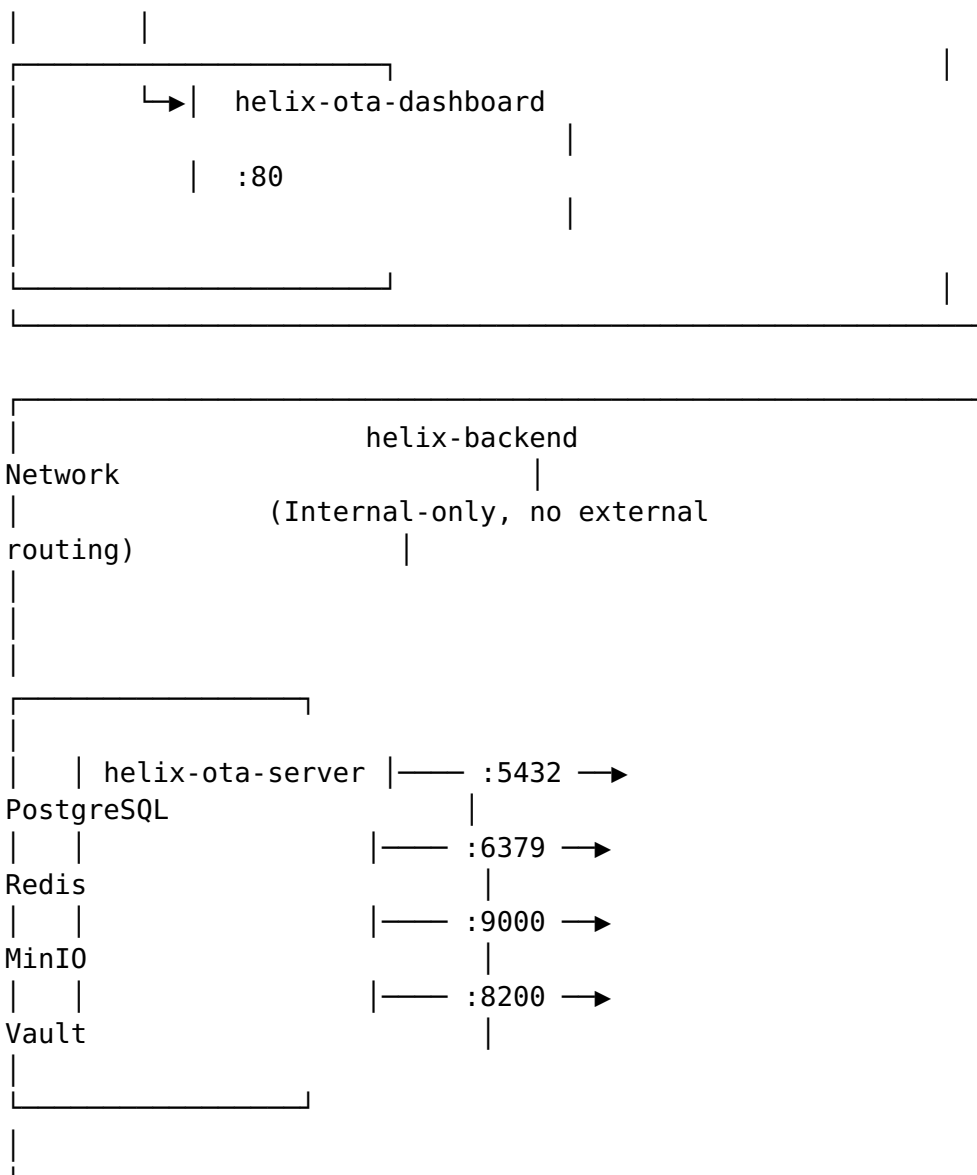
| Variable                  | Type   | Default                    | Required | Description                           |
|---------------------------|--------|----------------------------|----------|---------------------------------------|
| HELIX_METRICS_ENABLED     | bool   | true                       | No       | Enable Prometheus metrics endpoint    |
| HELIX_METRICS_PATH        | string | /metrics                   | No       | HTTP path for Prometheus metrics      |
| HELIX_TRACING_ENABLED     | bool   | false                      | No       | Enable OpenTelemetry tracing          |
| HELIX_TRACING_ENDPOINT    | string | http://otel-collector:4318 | No       | OTLP HTTP endpoint                    |
| HELIX_TRACING_SAMPLE_RATE | float  | 0.1                        | No       | Trace sampling rate (10% of requests) |

## 6. Networking

### 6.1 Internal Network Topology

The Helix OTA stack uses two isolated Docker networks (or Kubernetes NetworkPolicies) to enforce the principle of least privilege:





## 6.2 Port Mapping

| Service                  | Internal Port | External Port (Dev) | Protocol | Purpose                    |
|--------------------------|---------------|---------------------|----------|----------------------------|
| Traefik                  | 80, 443       | 80, 443             | HTTP(S)  | Public ingress             |
| Traefik Dashboard        | 8080          | 8080                | HTTP     | Admin dashboard (dev only) |
| helix-ota-server         | 8080          | 9090                | HTTP     | REST API                   |
| helix-ota-server (Delve) | 2345          | 2345                | TCP      | Go debugger (dev only)     |
|                          | 80            | 3000                | HTTP     | Web UI                     |

| Service             | Internal Port | External Port (Dev) | Protocol | Purpose                      |
|---------------------|---------------|---------------------|----------|------------------------------|
| helix-ota-dashboard |               |                     |          |                              |
| PostgreSQL          | 5432          | 5432                | TCP      | Database (dev only)          |
| Redis               | 6379          | 6379                | TCP      | Cache (dev only)             |
| MinIO API           | 9000          | 9000                | HTTP     | S3-compatible API (dev only) |
| MinIO Console       | 9001          | 9001                | HTTP     | Admin console (dev only)     |
| Vault               | 8200          | 8200                | HTTP     | Secret management (dev only) |

**Production rule:** Only Traefik ports 80 and 443 are exposed externally. All other services communicate exclusively over internal networks.

## 6.3 DNS and Service Discovery

| Hostname         | Resolves To         | Used By            | Purpose                  |
|------------------|---------------------|--------------------|--------------------------|
| api.helix-ota.io | Traefik → server    | Devices, Dashboard | REST API endpoint        |
| app.helix-ota.io | Traefik → dashboard | Dashboard users    | Web UI                   |
| s3.helix-ota.io  | Traefik → MinIO     | Devices (optional) | Direct artifact download |
| postgresql       | PostgreSQL service  | Server             | Database queries         |
| redis            | Redis service       | Server             | Cache operations         |
| minio            | MinIO service       | Server             | Artifact storage         |
| vault            | Vault service       | Server             | Secret access            |

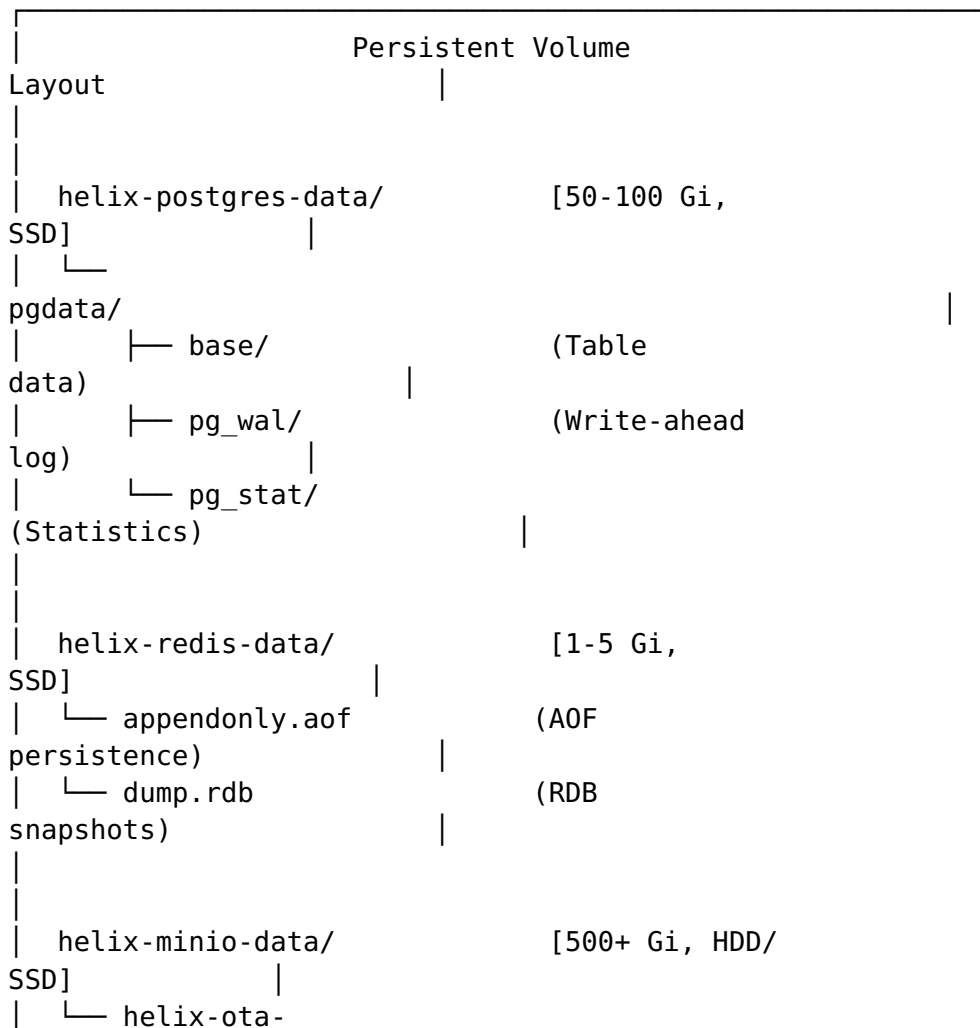
## 6.4 mTLS Configuration for Device Endpoints

Device-facing API endpoints require mutual TLS. Traefik is configured to pass through the client certificate to the server for CN extraction:

```
# Traefik mTLS configuration for device endpoints
labels:
- "traefik.http.routers.api-device.tls.options=mtls@file"
# Dynamic configuration file for mTLS:
# tls:
#   options:
#     mtls:
#       clientAuth:
#         ca: /etc/traefik/tls/device-ca.crt
#         clientAuthType: VerifyClientCertIfGiven
```

## 7. Storage

## 7.1 Volume Architecture



|                            |                 |  |
|----------------------------|-----------------|--|
| artifacts/                 |                 |  |
| artifacts/                 | (OTA zip        |  |
| binaries)                  |                 |  |
|                            |                 |  |
| art_01H.../                |                 |  |
|                            |                 |  |
| ota_update.zip             |                 |  |
|                            |                 |  |
| art_02H.../                |                 |  |
| signatures/                | (RSA-4096       |  |
| signatures)                |                 |  |
|                            |                 |  |
| helix-vault-data/          | [100 Mi-10 Gi,  |  |
| encrypted]                 |                 |  |
| raft/                      | (Raft consensus |  |
| storage)                   |                 |  |
|                            |                 |  |
| helix-traefik-letsencrypt/ | [50             |  |
| Mi]                        |                 |  |
| acme.json                  | (TLS            |  |
| certificates)              |                 |  |

## 7.2 Backup Strategy

| Data Store          | Backup Method                  | Frequency                           | Retention                         | Storage Location     | RTO     | RPO |
|---------------------|--------------------------------|-------------------------------------|-----------------------------------|----------------------|---------|-----|
| <b>PostgreSQL</b>   | pg_dump + WAL archiving        | Continuous WAL + Daily full         | 30 days incremental, 90 days full | MinIO (S3) + Offsite | 1 hour  | 0   |
| <b>Redis</b>        | AOF + RDB snapshots            | Every 60s (AOF) + Every 15min (RDB) | 7 days                            | Local volume         | 5 min   | 60s |
| <b>MinIO</b>        | MinIO replication + versioning | Continuous                          | 90 days versioning                | Cross-region MinIO   | 4 hours | 0   |
| <b>Vault</b>        | Raft snapshots                 | Daily                               | 30 days                           | Encrypted S3 bucket  | 30 min  | 2h  |
| <b>Config/Certs</b> | GitOps (declarative)           | On change                           | Indefinite                        | Git repository       | 15 min  | 0   |

## 7.3 PostgreSQL Backup Script

```
#!/bin/bash
# scripts/backup-postgres.sh
set -euo pipefail

BACKUP_DIR="/backups/postgresql"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
PG_HOST="${PG_HOST:-postgresql}"
PG_PORT="${PG_PORT:-5432}"
PG_USER="${PG_USER:-helix_ota}"
PG_DB="${PG_DB:-helix_ota}"
S3_BUCKET="${S3_BUCKET:-s3://helix-ota-backups/postgresql}"
RETENTION_DAYS="${RETENTION_DAYS:-30}"

mkdir -p "${BACKUP_DIR}"

# Full database dump with custom format (parallelizable restore)
echo "[$(date -Iseconds)] Starting PostgreSQL backup..."
pg_dump \
  -h "${PG_HOST}" \
  -p "${PG_PORT}" \
  -U "${PG_USER}" \
  -d "${PG_DB}" \
  --format=custom \
  --compress=9 \
  --verbose \
  --file="${BACKUP_DIR}/helix_ota_${TIMESTAMP}.dump"

# Upload to S3/MinIO
echo "[$(date -Iseconds)] Uploading to S3..."
aws s3 cp \
  "${BACKUP_DIR}/helix_ota_${TIMESTAMP}.dump" \
  "${S3_BUCKET}/helix_ota_${TIMESTAMP}.dump" \
  --storage-class STANDARD_IA

# Cleanup local backups older than retention period
find "${BACKUP_DIR}" -name "helix_ota_*.dump" -mtime +${RETENTION_DAYS} -delete

# Cleanup S3 backups older than retention period
aws s3 ls "${S3_BUCKET}/" | while read -r line; do
  createDate=$(echo "$line" | awk '{print $1" "$2}')
  createDate=$(date -d "$createDate" +%s)
  olderThan=$(date -d "-${RETENTION_DAYS} days" +%s)
  if [[ $createDate -lt $olderThan ]]; then
    fileName=$(echo "$line" | awk '{print $4}')
    aws s3 rm "${S3_BUCKET}/${fileName}"
  fi
done
```

```
echo "[$(date -Iseconds)] Backup completed: helix_ota_${TIMESTAMP}.dump"
```

## 7.4 PostgreSQL Restore Script

```
#!/bin/bash
# scripts/restore-postgres.sh
set -euo pipefail

BACKUP_FILE="${1:?Usage: $0 <backup-file-or-s3-url>}"
PG_HOST="${PG_HOST:-postgresql}"
PG_PORT="${PG_PORT:-5432}"
PG_USER="${PG_USER:-helix_ota}"
PG_DB="${PG_DB:-helix_ota}"

LOCAL_FILE="${BACKUP_FILE}"

# Download from S3 if needed
if [[ "${BACKUP_FILE}" == s3://* ]]; then
    echo "[$(date -Iseconds)] Downloading backup from S3..."
    LOCAL_FILE="/tmp/restore_$(date +%Y%m%d_%H%M%S).dump"
    aws s3 cp "${BACKUP_FILE}" "${LOCAL_FILE}"
fi

echo "[$(date -Iseconds)] WARNING: This will drop and
recreate the database!"
echo "[$(date -Iseconds)] Target: ${PG_HOST}:${PG_PORT}/${PG_DB}"
read -p "Continue? [y/N] " confirm
[[ "${confirm}" != "y" && "${confirm}" != "Y" ]] && exit 1

# Terminate existing connections
psql -h "${PG_HOST}" -U "${PG_USER}" -d postgres -c \
    "SELECT pg_terminate_backend(pid) FROM pg_stat_activity
    WHERE datname = '${PG_DB}' AND pid <>
    pg_backend_pid();"

# Drop and recreate database
psql -h "${PG_HOST}" -U "${PG_USER}" -d postgres -c "DROP
    DATABASE IF EXISTS ${PG_DB};"
psql -h "${PG_HOST}" -U "${PG_USER}" -d postgres -c "CREATE
    DATABASE ${PG_DB};"

# Restore from backup
echo "[$(date -Iseconds)] Restoring from ${LOCAL_FILE}..."
pg_restore \
    -h "${PG_HOST}" \
    -p "${PG_PORT}" \
    -U "${PG_USER}" \
    -d "${PG_DB}" \
    --verbose \
```

```

--no-owner \
--no-privileges \
--jobs=4 \
"${LOCAL_FILE}"

echo "[$(date -Iseconds)] Restore completed successfully."

# Cleanup
if [[ "${BACKUP_FILE}" == s3://* ]]; then
    rm -f "${LOCAL_FILE}"
fi

```

---

## 8. Scaling

### 8.1 Horizontal Scaling Strategy

The Helix OTA server is stateless (all state is in PostgreSQL, Redis, and MinIO) and can be horizontally scaled by adding replicas. The primary scaling dimensions are:

| Dimension                    | Trigger                                | Action                                   | Component        |
|------------------------------|----------------------------------------|------------------------------------------|------------------|
| <b>API Throughput</b>        | CPU > 70%<br>or latency<br>p95 > 500ms | Add server replicas (HPA)                | helix-ota-server |
| <b>Database Load</b>         | Active connections > 80% of pool       | Add read replicas; increase pool size    | PostgreSQL       |
| <b>Cache Hit Rate</b>        | Hit rate < 90%                         | Increase Redis memory; extend TTL        | Redis            |
| <b>Artifact I/O</b>          | Download latency > 2s                  | Add MinIO nodes; enable CDN              | MinIO / CDN      |
| <b>WebSocket Connections</b> | > 10K concurrent dashboard sessions    | Add server replicas; use sticky sessions | helix-ota-server |

## 8.2 Kubernetes HPA Configuration

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: helix-ota-server-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: helix-ota-server
  minReplicas: 3
  maxReplicas: 50
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second
        target:
          type: AverageValue
          averageValue: "1000"
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      selectPolicy: Max
      policies:
        - type: Percent
          value: 100
          periodSeconds: 30
        - type: Pods
          value: 5
          periodSeconds: 30
    scaleDown:
      stabilizationWindowSeconds: 300
      selectPolicy: Min
      policies:
        - type: Percent
          value: 25
          periodSeconds: 60
```

### 8.3 Resource Limits Summary

| Service              | CPU Request | CPU Limit | Memory Request | Memory Limit | Max Replicas     |
|----------------------|-------------|-----------|----------------|--------------|------------------|
| helix-ota-server     | 500m        | 2         | 256Mi          | 1Gi          | 50               |
| helix-ota-dashboard  | 100m        | 250m      | 64Mi           | 128Mi        | 4                |
| PostgreSQL (primary) | 1           | 4         | 2Gi            | 8Gi          | 1 (+ 2 replicas) |
| Redis (master)       | 250m        | 1         | 256Mi          | 1Gi          | 1 (+ 2 replicas) |
| MinIO                | 500m        | 2         | 512Mi          | 2Gi          | 4 (distributed)  |
| Vault                | 250m        | 1         | 256Mi          | 512Mi        | 3 (HA)           |
| Traefik              | 250m        | 1         | 128Mi          | 512Mi        | 3                |

### 8.4 Scaling for Fleet Size

| Fleet Size       | Server Replicas | PG Cores | PG Memory | Redis Memory | MinIO Nodes   |
|------------------|-----------------|----------|-----------|--------------|---------------|
| < 1,000 devices  | 2               | 2        | 4 Gi      | 256 Mi       | 1             |
| 1,000 - 10,000   | 3-5             | 4        | 8 Gi      | 512 Mi       | 2             |
| 10,000 - 100,000 | 5-15            | 8        | 16 Gi     | 1 Gi         | 4             |
| 100,000+         | 15-50           | 16       | 32 Gi     | 2 Gi         | 4+ (with CDN) |

## 9. Monitoring

### 9.1 Prometheus Metrics Endpoints

| Service          | Endpoint      | Scrape Interval | Key Metrics                                                                                                                                           |
|------------------|---------------|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| helix-ota-server | :8080/metrics | 15s             | http_requests_total, http_request_duration_seconds, artifact_uploads_total, rollout_active_count, device_checkin, db_pool_connections, cache_hit_rate |

| Service    | Endpoint                       | Scrape Interval | Key Metrics                                                                                                             |
|------------|--------------------------------|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| PostgreSQL | :9187/metrics (pg_exporter)    | 30s             | pg_stat_database_numbackends,<br>pg_stat_database_xact_commit,<br>pg_stat_activity_count,<br>pg_replication_lag_seconds |
| Redis      | :9121/metrics (redis_exporter) | 15s             | redis_connected_clients,<br>redis_used_memory_bytes,<br>redis_commands_processed_total,<br>redis_keyspace_hits_total    |
| MinIO      | :9000/minio/v2/metrics/cluster | 30s             | minio_bucket_usage_total_bytes,<br>minio_s3_requests_total,<br>minio_s3_errors_total                                    |
| Vault      | :8200/v1/sys/metrics           | 30s             | vault_core_handle_count,<br>vault_raft_commit_time,<br>vault_token_create_count                                         |
| Traefik    | :8080/metrics                  | 15s             | traefik_entrypoint_requests_total,<br>traefik_entrypoint_request_duration,<br>traefik_router_requests_total             |

## 9.2 Custom Application Metrics

The Helix OTA server exposes the following custom Prometheus metrics through the vasic-digital/observability submodule:

```
// Server-side custom metrics (registered at startup)
var (
    // Update check metrics
    updateCheckTotal =
        promauto.NewCounterVec(prometheus.CounterOpts{
            Name: "helix_update_check_total",
            Help: "Total number of device update check requests",
        }, []string{"device_group", "result"}) // result:
        "update_available", "no_update", "error"

    updateCheckDuration =
        promauto.NewHistogramVec(prometheus.HistogramOpts{
            Name:    "helix_update_check_duration_seconds",
            Help:    "Duration of update check processing",
            Buckets: prometheus.ExponentialBuckets(0.001, 2,
                15), // 1ms to ~16s
        }, []string{"device_group"})

    // Artifact metrics
    artifactUploadTotal =
        promauto.NewCounterVec(prometheus.CounterOpts{
            Name: "helix_artifact_upload_total",
            Help: "Total artifact upload attempts",
```

```

}, []string{"status"}) // status: "success",
                        "validation_failed", "storage_error"

artifactUploadSize =
    promauto.NewHistogram(prometheus.HistogramOpts{
        Name:    "helix_artifact_upload_bytes",
        Help:    "Size of uploaded artifacts",
        Buckets: prometheus.ExponentialBuckets(1048576, 2,
        12), // 1MB to 2GB
    })

artifactValidationDuration =
    promauto.NewHistogramVec(prometheus.HistogramOpts{
        Name:    "helix_artifact_validation_duration_seconds",
        Help:    "Duration of artifact validation chain",
        Buckets: prometheus.ExponentialBuckets(0.1, 2, 10),
    }, []string{"stage", "result"})

// Rollout metrics
rolloutActiveGauge =
    promauto.NewGaugeVec(prometheus.GaugeOpts{
        Name: "helix_rollout_active_count",
        Help: "Number of currently active rollouts",
    }, []string{"strategy", "device_group"})

rolloutPercentageGauge =
    promauto.NewGaugeVec(prometheus.GaugeOpts{
        Name: "helix_rollout_current_percentage",
        Help: "Current rollout percentage",
    }, []string{"rollout_id"})

// Device metrics
deviceCheckinTotal =
    promauto.NewCounterVec(prometheus.CounterOpts{
        Name: "helix_device_checkin_total",
        Help: "Total device check-in events",
    }, []string{"device_group", "status"})

deviceUpdateStatus =
    promauto.NewGaugeVec(prometheus.GaugeOpts{
        Name: "helix_device_update_status_count",
        Help: "Count of devices by update status",
    }, []string{"status"}) // pending, downloading,
                           verifying, installing, succeeded, failed

// Telemetry metrics
telemetryIngestTotal =
    promauto.NewCounterVec(prometheus.CounterOpts{
        Name: "helix_telemetry_ingest_total",
        Help: "Total telemetry events ingested",
    }, []string{"event_type"})

```

```

    anomalyDetectedTotal =
        promauto.NewCounterVec(prometheus.CounterOpts{
            Name: "helix_anomaly_detected_total",
            Help: "Total anomaly events detected",
        }, []string{"anomaly_type"})
)

```

## 9.3 Grafana Dashboard Definitions

The following dashboards are provisioned automatically via ConfigMap:

| Dashboard                  | Panels                                                                     | Refresh Interval | Purpose               |
|----------------------------|----------------------------------------------------------------------------|------------------|-----------------------|
| <b>Helix OTA Overview</b>  | Fleet size, active rollouts, update success rate, API latency p95          | 30s              | Executive overview    |
| <b>Device Fleet Health</b> | Devices by status, check-in rate, stale devices, version distribution      | 1m               | Fleet operations      |
| <b>Rollout Progress</b>    | Active rollouts, percentage per rollout, failure rate, auto-advance events | 15s              | Rollout monitoring    |
| <b>Artifact Pipeline</b>   | Upload rate, validation duration, storage usage by artifact                | 1m               | Artifact management   |
| <b>Infrastructure</b>      | PostgreSQL connections, query latency, Redis hit rate, MinIO I/O           | 30s              | Infrastructure health |

| Dashboard | Panels                                                                           | Refresh Interval | Purpose             |
|-----------|----------------------------------------------------------------------------------|------------------|---------------------|
| Security  | Auth failures, rate limit rejections, certificate renewal status, anomaly alerts | 1m               | Security monitoring |

## 9.4 Alerting Rules

*# Prometheus alerting rules for Helix OTA*

groups:

- name: helix-ota-alerts
 rules:
  - alert: HelixOTAServerHighErrorRate
 expr: rate(helix\_update\_check\_total{result="error"}[5m]) / rate(helix\_update\_check\_total[5m]) > 0.05
 for: 5m
 labels:
 severity: warning
 annotations:
 summary: "High error rate on update checks"
 description:
 "{{ \$value | humanizePercentage }}" of update checks are failing"
  - alert: HelixOTARolloutHighFailureRate
 expr:
 helix\_device\_update\_status\_count{status="failed"} /
 (helix\_device\_update\_status\_count{status="succeeded"}
 + helix\_device\_update\_status\_count{status="failed"}) >
 0.05
 for: 10m
 labels:
 severity: critical
 annotations:
 summary: "Rollout failure rate exceeds 5% threshold"
 description: "Failure rate is {{ \$value | humanizePercentage }}"
  - alert: HelixOTAPostgreSQLConnectionPoolExhaustion
 expr: db\_pool\_connections{state="idle"} < 5
 for: 5m
 labels:
 severity: critical
 annotations:

```
summary: "PostgreSQL connection pool nearly
exhausted"

- alert: HelixOTARedisCacheHitRateLow
  expr: rate(redis_keyspace_hits_total[5m]) /
    (rate(redis_keyspace_hits_total[5m]) +
    rate(redis_keyspace_misses_total[5m])) < 0.80
  for: 15m
  labels:
    severity: warning
  annotations:
    summary: "Redis cache hit rate below 80%"

- alert: HelixOTAMinIOStorageHigh
  expr: minio_bucket_usage_total_bytes / (1024 * 1024
    * 1024) > 450
  for: 1h
  labels:
    severity: warning
  annotations:
    summary: "MinIO storage exceeds 450 GiB (90% of
    500 GiB)"

- alert: HelixOTADevicesStale
  expr:
    count(helix_device_checkin_total{status="offline"}
    > bool 0) > 100
  for: 2h
  labels:
    severity: warning
  annotations:
    summary:
      "More than 100 devices are offline for >2 hours"

- alert: HelixOTAVaultSealed
  expr: vault_core_unsealed == 0
  for: 1m
  labels:
    severity: critical
  annotations:
    summary: "Vault is sealed – signing operations
    will fail"
```

---

# 10. Disaster Recovery

## 10.1 Recovery Time and Point Objectives

| Scenario                            | RTO                | RPO                     | Recovery Procedure                                                  |
|-------------------------------------|--------------------|-------------------------|---------------------------------------------------------------------|
| <b>Server pod crash</b>             | 30s                | 0                       | Kubernetes auto-restarts pod; no data loss (stateless)              |
| <b>PostgreSQL primary failure</b>   | 5 min              | 0 (synchronous replica) | Promote read replica; update service DNS                            |
| <b>Redis master failure</b>         | 1 min              | 60s (AOF)               | Sentinel promotes replica; cold cache rebuilds from DB              |
| <b>MinIO node failure</b>           | 0 (if distributed) | 0                       | Distributed mode continues serving; replace failed node             |
| <b>Vault seal/crash</b>             | 5 min              | 0                       | Auto-unshard with Kubernetes secret; Raft log replay                |
| <b>Complete cluster loss</b>        | 4 hours            | 24 hours                | Restore PostgreSQL from S3 backup; re-provision cluster from GitOps |
| <b>Ransomware / data corruption</b> | 8 hours            | 24 hours                | Point-in-time recovery from WAL                                     |

| Scenario | RTO | RPO | Recovery Procedure                       |
|----------|-----|-----|------------------------------------------|
|          |     |     | archive;<br>verify<br>artifact<br>hashes |

## 10.2 Complete Disaster Recovery Runbook

```
#!/bin/bash
# scripts/disaster-recovery.sh
# Complete cluster recovery from bare metal
set -euo pipefail

echo "=== Helix OTA Disaster Recovery Procedure ==="
echo "Started at: $(date -Iseconds)"

# Phase 1: Infrastructure Provisioning
echo "[Phase 1] Provisioning Kubernetes cluster..."
echo "  - Create cluster: kubectl apply -f infrastructure/"
echo "  - Install cert-manager: kubectl apply -f cert-manager/"
echo "  - Install Traefik: helm install traefik traefik/traefik"

# Phase 2: Secrets Recovery
echo "[Phase 2] Restoring secrets..."
echo "  - Restore Vault unseal keys from offline backup"
echo "  - kubectl create secret generic helix-ota-vault-unseal --from-literal=..."
echo "  - Restore database credentials"
echo "  - Restore MinIO credentials"

# Phase 3: Database Recovery
echo "[Phase 3] Restoring PostgreSQL..."
LATEST_BACKUP=$(aws s3 ls s3://helix-ota-backups/postgresql/ | sort | tail -1 | awk '{print $4}')
echo "  - Latest backup: ${LATEST_BACKUP}"
echo "  - Downloading: aws s3 cp s3://helix-ota-backups/postgresql/${LATEST_BACKUP} /tmp/"
echo "  - Restoring: ./scripts/restore-postgres.sh /tmp/${LATEST_BACKUP}"

# Phase 4: Object Storage Recovery
echo "[Phase 4] Restoring MinIO artifacts..."
echo "  - MinIO data restored from cross-region replication"
echo "  - Verify artifact count: mc ls helix-minio/helix-ota-artifacts/artifacts/ | wc -l"
echo "  - Verify artifact integrity: check SHA-256 hashes against database records"

# Phase 5: Application Deployment
```

```

echo "[Phase 5] Deploying Helix OTA services..."
echo "  - helm install helix-ota ./helm/helix-ota -f values-
    production.yaml"
echo "  - Wait for all pods: kubectl wait --
    for=condition=ready pod -l app.kubernetes.io/part-
    of=helix-ota"

# Phase 6: Verification
echo "[Phase 6] Running health checks..."
echo "  - API health: curl -sf https://api.helix-ota.io/
    healthz"
echo "  - Database: kubectl exec -it postgresql-0 --
    pg_isready"
echo "  - Redis: kubectl exec -it redis-0 -- redis-cli ping"
echo "  - MinIO: curl -sf https://s3.helix-ota.io/minio/
    health/live"
echo "  - Vault: kubectl exec -it vault-0 -- vault status"

echo "=== Disaster Recovery Complete ==="
echo "Completed at: $(date -Iseconds)"

```

## 10.3 Backup Verification Schedule

| Check                                      | Frequency    | Automated | Alerting             |
|--------------------------------------------|--------------|-----------|----------------------|
| PostgreSQL backup exists in S3             | Daily        | Yes       | PagerDuty if missing |
| WAL archiving lag < 60s                    | Every 5 min  | Yes       | PagerDuty if lagging |
| MinIO cross-region replication lag < 5 min | Every 15 min | Yes       | Slack notification   |
| Vault Raft snapshot < 24 hours old         | Daily        | Yes       | PagerDuty if stale   |
| Full restore test (staging)                | Weekly       | Yes       | Slack notification   |
| Full restore test (production)             | Monthly      | Manual    | Runbook verification |

# 11. vasic-digital/containers Integration

## 11.1 Submodule Architecture

The vasic-digital/containers submodule provides the container orchestration foundation for Helix OTA. It is included as a Git submodule at /containers and provides standardized container definitions, Docker Compose templates, and Kubernetes manifests that are shared across all vasic-digital projects.

```
helix-ota/
├── containers/                                # Git submodule: vasic-
digital/containers
├── ├── docker-compose.yml                    # Main compose file (Helix
OTA customized)
├── ├── docker-compose.prod.yml              # Production overrides
├── ├── docker-compose.dev.yml              # Development overrides
(hot-reload, debug)
├── ├── Dockerfiles/
├── │   ├── server.Dockerfile                # Multi-stage Go build
├── │   └── dashboard.Dockerfile            # Multi-stage React +
nginx build
├── │   ├── init-db/
├── │   │   ├── 001_extensions.sql           # Enable pgcrypto, UUID
├── │   │   ├── 002_enum_types.sql          # All enum definitions
├── │   │   ├── 003_core_tables.sql         # All table definitions
├── │   │   ├── 004_indexes.sql             # All index definitions
├── │   │   └── 005_seed_data.sql           # Initial admin user,
device groups
├── │   ├── vault-config/
├── │   │   ├── development.hcl             # Dev Vault config
├── │   │   └── production.hcl              # Prod Vault config (Raft
+ TLS)
├── │   ├── scripts/
├── │   │   ├── backup-postgres.sh
├── │   │   ├── restore-postgres.sh
├── │   │   ├── disaster-recovery.sh
├── │   │   ├── minio-init.sh
├── │   │   └── vault-init.sh
├── │   └── helm/
├── │       └── helix-ota/                    # Helm chart (described in
$4)
└── server/                                # Go source code
```

## 11.2 Initializing the Submodule

To set up the containers submodule in a fresh clone:

```
# Clone with submodules
git clone --recurse-submodules https://github.com/vasic-digital/helix-ota.git

# Or initialize after clone
cd helix-ota
git submodule update --init --recursive containers

# Pull latest submodule updates
git submodule update --remote containers
```

## 11.3 Submodule vs. Project Responsibilities

The vasic-digital/containers submodule provides **infrastructure templates** and **shared tooling**. Project-specific customization happens in the parent repository through Docker Compose overrides and Helm value files.

| Responsibility                | Submodule (containers/)       | Project (helix-ota/)                     |
|-------------------------------|-------------------------------|------------------------------------------|
| Base Docker Compose structure | Provides                      | Customizes with environment variables    |
| Dockerfile templates          | Provides multi-stage patterns | Customizes build args, base images       |
| Database init scripts         | Provides schema DDL           | Provides seed data specific to Helix OTA |
| Vault configuration           | Provides HCL templates        | Customizes secrets paths, policies       |
| Helm chart structure          | Provides chart skeleton       | Customizes values.yaml for Helix OTA     |
| Backup/restore scripts        | Provides templates            | Customizes connection strings, S3 paths  |
| Network definitions           | Provides base networks        | Customizes subnets, labels               |
| Volume definitions            | Provides base volumes         | Customizes sizes, storage classes        |

## 11.4 Submodule-Provided vasic-digital Libraries

The containers submodule integrates the following vasic-digital Go libraries that are imported directly by the Helix OTA server:

| Submodule Library           | Container Integration                        | Purpose                                         |
|-----------------------------|----------------------------------------------|-------------------------------------------------|
| vasic-digital/auth          | Vault Transit for JWT signing                | JWT token creation, validation, rotation        |
| vasic-digital/database      | PostgreSQL connection via HELIX_DATABASE_URL | pgxpool connection management, migration runner |
| vasic-digital/cache         | Redis connection via HELIX_REDIS_URL         | L1/L2 caching with TTL, invalidation            |
| vasic-digital/observability | Prometheus endpoint at :8080/metrics         | Metrics, structured logging, tracing            |
| vasic-digital/storage       | MinIO connection via HELIX_MINIO_*           | S3-compatible multipart upload/download         |
| vasic-digital/security      | Vault + TLS cert mounts                      | TLS configuration, certificate management       |
| vasic-digital/config        | Environment variable loading                 | Config struct hydration from HELIX_* vars       |
| vasic-digital/middleware    | Server middleware chain                      | Request ID, CORS, auth, logging                 |
| vasic-digital/ratelimiter   | Redis-backed token bucket                    | Per-device, per-IP, per-user rate limiting      |
| vasic-digital/EventBus      | In-process event dispatch                    | Async event processing                          |

| Submodule Library         | Container Integration      | Purpose                                   |
|---------------------------|----------------------------|-------------------------------------------|
|                           |                            | for telemetry, rollouts                   |
| vasic-digital/concurrency | Worker pool for validation | Goroutine pool with cancellation          |
| vasic-digital/recovery    | Panic recovery middleware  | Graceful degradation on unexpected errors |

## 11.5 Updating the Submodule

```
# Check for submodule updates
cd containers
git fetch origin
git log HEAD..origin/main --oneline

# Update to latest main
git pull origin main
cd ..
git add containers
git commit -m "chore: update vasic-digital/containers submodule"

# Pin to a specific release tag (recommended for production)
cd containers
git checkout v1.2.3
cd ..
git add containers
git commit -m "chore: pin containers submodule to v1.2.3"
```

## 11.6 Development Workflow with Docker Compose

```
# Start the complete stack
cd containers
docker compose up -d

# Start with development overrides (hot-reload, debug ports)
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d

# View logs for a specific service
docker compose logs -f helix-ota-server

# Run database migrations
```

```
docker compose exec helix-ota-server /app/migrate up

# Access PostgreSQL directly
docker compose exec postgresql psql -U helix_ota -d
helix_ota

# Access Redis CLI
docker compose exec redis redis-cli -a "${REDIS_PASSWORD}"

# Access MinIO admin console
open https://localhost:9001

# Stop all services and preserve data
docker compose down

# Stop and remove all data (clean slate)
docker compose down -v

# Rebuild server image after code changes
docker compose build helix-ota-server

# Scale server replicas (load testing)
docker compose up -d --scale helix-ota-server=3
```

## 11.7 Production Deployment Checklist

Before deploying to production, verify every item:

☐

**Secrets:** All passwords, keys, and tokens stored in Vault (not environment variables)

☐

**TLS:** All inter-service communication uses TLS; only Traefik exposes HTTP

☐

**mTLS:** Device CA certificate mounted; CRL endpoint configured

☐

**Backups:** PostgreSQL WAL archiving verified; MinIO replication confirmed

☐

**Monitoring:** Prometheus scraping all endpoints; Grafana dashboards provisioned

☐

**Alerting:** PagerDuty/Slack integration tested with test alerts

☐

**Scaling:** HPA configured and tested with load test (k6)

☐

- ☐ **Network Policies:** Kubernetes NetworkPolicies applied; only approved traffic flows
- ☐ **Resource Limits:** All containers have CPU and memory limits set
- ☐ **Image Security:** Trivy scan passed with zero CRITICAL/HIGH CVEs
- ☐ **Database:** Migrations run successfully; seed data verified
- ☐ **Health Checks:** All services have liveness and readiness probes
- ☐ **Vault:** Transit engine initialized; artifact signing key created; auto-unseal configured
- ☐ **MinIO:** Bucket created with versioning enabled; lifecycle policy configured
- ☐ **DNS:** All hostnames resolve correctly; TLS certificates provisioned
- ☐ **Disaster Recovery:** Backup restore tested on staging within last 30 days
- ☐ **Runbook:** On-call engineer has access to disaster recovery runbook
- ☐ **Submodule:** vasic-digital/containers pinned to release tag (not main)

---

Document generated as part of the Helix OTA 1.0.0-MVP documentation suite. For questions, contact the Helix OTA Engineering Team.